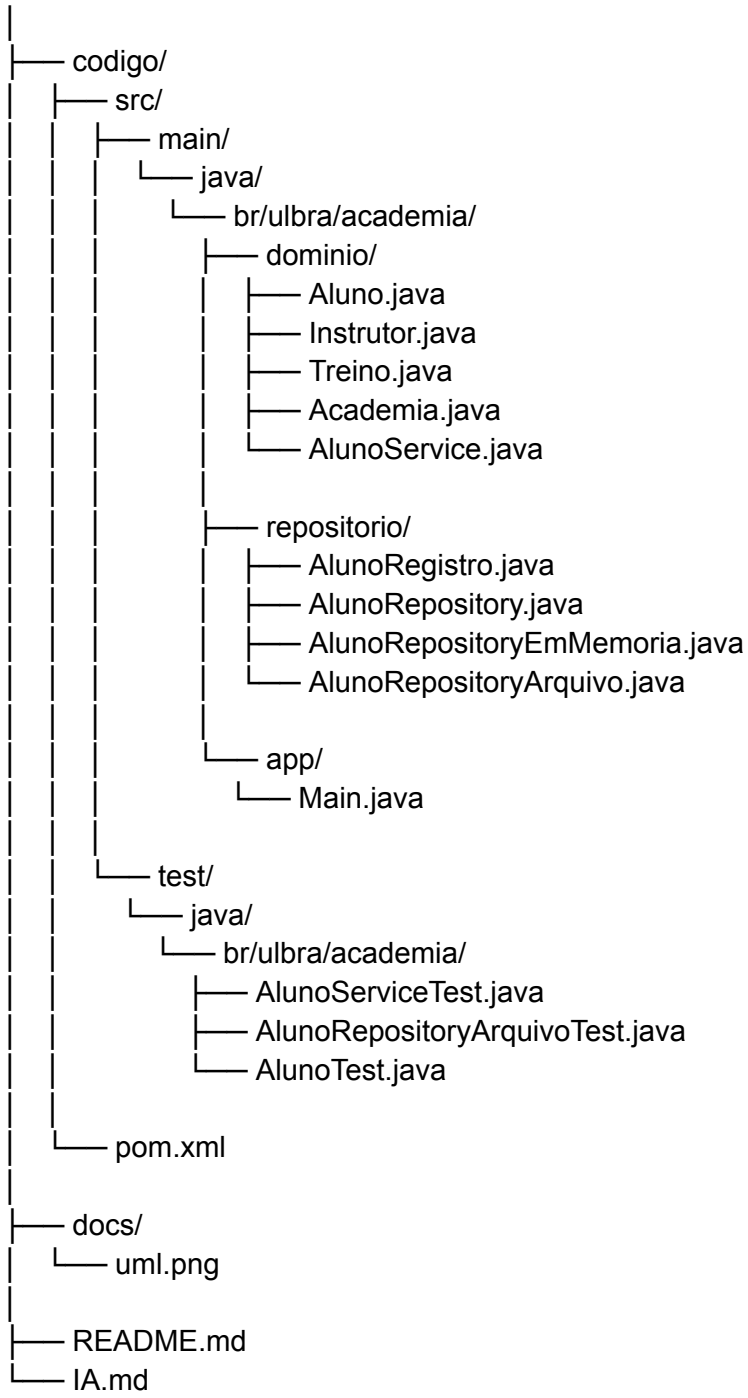
1. Estrutura de pastas

Vamos utilizar esta organização:



Importante: essa é a estrutura proposta. Precisamos preservar a estrutura de compilação e os arquivos que já existem na versão v11, sem apagar funcionalidades anteriores.

2. O que cada parte fará

Classe	Responsabilidade
`Aluno`	Representar o aluno
`Treino`	Representar um treino
`Instrutor`	Representar o instrutor
`Academia`	Organizar alunos e instrutores
`AlunoService`	Executar operações e regras de negócio
`AlunoRegistro`	Guardar os dados simples de um aluno
`AlunoRepository`	Definir como salvar e listar registros
`AlunoRepositoryEmMemoria`	Guardar registros durante a execução
`AlunoRepositoryArquivo`	Guardar registros em arquivo
`Main`	Escolher a implementação e executar o programa

A regra de negócio que vamos preservar como exemplo é: um aluno pode possuir vários treinos associados.

3. Primeira implementação: interface do repositório

Crie `AlunoRegistro.java`:

```
package br.ulbra.academia.repositorio;

public record AlunoRegistro(
    int id,
    String nome,
    int idade,
    String telefone
) {}
```

Agora crie `AlunoRepository.java`:

```
package br.ulbra.academia.repositorio;

import java.util.List;

public interface AlunoRepository {

    void salvar(AlunoRegistro aluno);

    List<AlunoRegistro> listarTodos();
}
```

Essa interface define duas operações:

- `salvar()`: guarda um aluno.
- `listarTodos()`: retorna os alunos cadastrados.

As duas implementações deverão cumprir esse contrato.

4. Como funcionará a persistência

Arquitetura do projeto

Main.java

Escolhe a implementação

AlunoService.java

Recebe AlunoRepository pelo construtor

AlunoRepository (interface)

Em memória

Dados temporários

Em arquivo

Dados persistentes

5. Preciso conferir o código original

Consegui localizar o repositório e confirmar que ele já possui a pasta `codigo`, documentação e README.

GitHub

Entretanto, não consegui abrir os arquivos Java individualmente para verificar a implementação atual. Não quero te entregar códigos que entrem em conflito com o projeto de vocês.

Você consegue conectar o GitHub pelo recurso que apareceu aqui ou prefere enviar o projeto em um arquivo ZIP?

Assim consigo trabalhar com a estrutura real e preparar a versão `v12`, incluindo os testes, o UML, o README e os comandos para publicar a tag.

Lembre-se: a entrega é no dia 09/11, e precisamos conferir especialmente a persistência entre duas execuções e os quatro testes JUnit.

Também posso te ajudar a conferir os testes e a tag antes da entrega.

O próximo passo é acessar o código da `v11` para adaptar tudo corretamente. Não consegui confirmar os arquivos do projeto nesta tentativa.

Envie o ZIP do repositório aqui na conversa. Com ele, consigo analisar o código existente e preparar as alterações sem substituir funcionalidades que vocês já fizeram.

Não vou considerar a `v12` pronta até conferir os quatro testes, as duas implementações de persistência, o UML e o README.